🚀 Guía e Instrucciones de Ejecución del Proyecto (Linux / WSL2 / Windows y Docker)

Este documento detalla paso a paso cómo configurar, inicializar y ejecutar todos los componentes del proyecto **Detección Automatizada de Fraude, Descubrimiento de Anomalías y Reglas de Decisión en Tiempo Real (KDD)**, además de documentar la causa raíz y solución al problema del bucle infinito al iniciar Docker en Linux/WSL2.

🛠️ 1. Diagnóstico y Solución: ¿Por qué Docker entraba en un bucle en Linux / WSL2?

Al iniciar el clúster de Apache Airflow en Linux o WSL2, el sistema parecía quedarse atrapado en un ciclo interminable por la combinación de **5 factores críticos** que ya han sido corregidos en el repositorio:

1. **Carga masiva de +50 DAGs de ejemplo (`AIRFLOW__CORE__LOAD_EXAMPLES: 'true'`):**
   - Tanto en `airflow/docker-compose.yaml` como en `airflow/config/airflow.cfg` estaba habilitada la carga de DAGs de prueba de Airflow.
   - Varios de estos DAGs de ejemplo se ejecutaban automáticamente cada minuto (`* * * * *`) y fallaban de forma continua, provocando que `airflow-dag-processor`, `airflow-scheduler` y `airflow-worker` entraran en un bucle perpetuo procesando archivos y escribiendo miles de líneas de log por minuto.
   - **Solución aplicada:** Se configuró `AIRFLOW__CORE__LOAD_EXAMPLES: 'false'` y `load_examples = False`, dejando únicamente los 2 DAGs reales del proyecto (`fraud_simulation_pipeline` y `fraud_model_retraining_pipeline`), y se incrementó el intervalo de escaneo (`MIN_FILE_PROCESS_INTERVAL: '60'`) para no saturar el disco en WSL2 (`/mnt/c`).

2. **Reinstalación de paquetes en cada arranque (`_PIP_ADDITIONAL_REQUIREMENTS`):**
   - Definir `_PIP_ADDITIONAL_REQUIREMENTS: "scikit-learn joblib pandas"` obligaba a los 5 contenedores de Airflow a descargar e instalar librerías pesadas desde internet **en cada inicio**, haciendo que los `healthcheck` expiraran y Docker reiniciara los contenedores en bucle.
   - **Solución aplicada:** La imagen oficial `apache/airflow:3.3.2` **ya incluye preinstalados** `scikit-learn`, `joblib`, `pandas`, `numpy`, `sqlalchemy` y `psycopg2`, por lo que se dejó `_PIP_ADDITIONAL_REQUIREMENTS` vacío para que los contenedores arranquen en pocos segundos.

3. **Conflicto entre archivo `.env` y directorio `.venv`:**
   - Originalmente `docker-compose.yaml` apuntaba a `env_file: .venv`. Al crear un entorno virtual de Python en Linux (`airflow/.venv`), Docker intentaba leer un directorio como archivo de variables de entorno, o bien `airflow-init` tardaba minutos ejecutando `chown -v -R` sobre miles de archivos.
   - **Solución aplicada:** Se fijó `env_file: .env`, se normalizaron los saltos de línea a `LF` (`.gitattributes`), se eliminó el flag verboso `-v` del `chown` en `airflow-init` y se excluyeron

`airflow/logs/*` y `airflow/.venv/` en `.gitignore`.

4. **Resolución de red en Linux (`host.docker.internal`) y volumen `/opt/airflow/models`:**
   - En Linux nativo, Docker no resuelve `host.docker.internal` a menos que se declare explícitamente, y al editar el `docker-compose.yaml` se había omitido el montaje de `/opt/airflow/models` (causando `FileNotFoundError: '/opt/airflow/models/fraud_classifier.joblib'` cada 5 minutos).
   - **Solución aplicada:** Se agregó `extra_hosts: ["host.docker.internal:host-gateway"]`, se restauró el volumen `- ${AIRFLOW_PROJ_DIR:-.}/models:/opt/airflow/models` y la variable `AIRFLOW_CONN_POSTGRES_DEFAULT`.
5. **Ejecución en primer plano (`up`) vs. segundo plano (`up -d`):**
   - Ejecutar `docker compose up` sin `-d` deja la terminal adjunta a la salida estándar de los 7 contenedores indefinidamente. Siempre debe usarse `docker compose -f airflow/docker-compose.yaml up -d`.

📋 2. Requisitos Previos del Sistema

- **Python:** `3.10`, `3.11` o `3.12`
- **PostgreSQL:** `15`, `16` o superior (corriendo en el puerto `5432`)
- **Docker y Docker Compose v2:**
  - En **Linux:** Docker Engine + `docker-compose-plugin` (mínimo 4 GB de RAM libres).
  - En **Windows:** Docker Desktop con motor WSL2 activado.

⚙️ 3. Paso a Paso para Ejecutar el Proyecto

Paso 1: Configurar el Entorno Virtual de Python e Instalar Dependencias

Abre una terminal en la carpeta raíz del proyecto:

🐧 En Linux / WSL2:

```
# 1. Crear el entorno virtual en la raíz del proyecto
python3 -m venv .venv

# 2. Activar el entorno virtual
source .venv/bin/activate

# 3. Actualizar pip e instalar las dependencias de Linux
pip install --upgrade pip
pip install -r requirements-linux.txt
```

🪟 En Windows (PowerShell):

```
# 1. Crear el entorno virtual (si en .venv existe uno de Linux, bórralo primero o
usa .venv-win)
python -m venv .venv

# 2. Activar el entorno virtual
.\.venv\Scripts\Activate.ps1

# 3. Instalar las dependencias de Windows
pip install --upgrade pip
pip install -r requirements-windows.txt
```

Paso 2: Inicializar y Poblar la Base de Datos PostgreSQL (100,000 Transacciones)

El proyecto incluye el script automatizado `sql/seed_database.py` que crea la base de datos `fraud_detection_kdd`, los esquemas (`raw`, `analytics`, `ml_ops`), las tablas, las vistas del *Feature Store* y carga las 100,000 transacciones del archivo `Datasets/credit_card_fraud_detection_2026.csv`.

```
python sql/seed_database.py \
  --host localhost \
  --port 5432 \
  --user postgres \
  --password TU_PASSWORD_POSTGRES \
  --dbname fraud_detection_kdd
```

> **Nota sobre credenciales:** Si tu contraseña de PostgreSQL es distinta a la configurada por defecto, puedes definir las variables de entorno antes de ejecutar los scripts:
>
> ```
> export PGHOST=localhost
> export PGPORT=5432
> export PGUSER=postgres
> export PGPASSWORD=TU_PASSWORD_POSTGRES
> export PGDATABASE=fraud_detection_kdd
> ```

Para verificar que la conexión a PostgreSQL funciona correctamente:

```
python src/db/db_connector.py
```

Paso 3: Verificar o Entrenar los Modelos de Machine Learning (KDD)

El repositorio **ya incluye los modelos pre-entrenados** listos para inferencia en las carpetas `models/` y `airflow/models/`:

- `fraud_classifier.joblib` (Random Forest calibrado con SMOTE)
- `isolation_forest.joblib` (Detector no supervisado de anomalías)
- `scaler.joblib` (`RobustScaler`)
- `model_columns.json` (Vector canónico de 45 características)

Si deseas **re-ejecutar el pipeline completo de las 8 etapas KDD** desde cero y regenerar los modelos y gráficas en `results/`:

```
python src/train_and_save.py
```

Paso 4: Iniciar el Clúster de Apache Airflow en Docker (Sin Bucles)

Desde la raíz del proyecto, sigue estos comandos:

1. (Solo en Linux) Verificar el `AIRFLOW_UID` en `airflow/.env`:

El archivo `airflow/.env` ya viene preconfigurado con `AIRFLOW_UID=1000` (el UID estándar del primer usuario en Ubuntu/Debian/WSL2). Si tu usuario en Linux tiene otro UID, puedes verificarlo con `id -u`.

2. Levantar los contenedores en segundo plano (`-d`):

```
docker compose -f airflow/docker-compose.yaml up -d
```

> **¿Qué ocurre al ejecutar este comando?**
>
> 1. Se levantan `postgres` (metadatos de Airflow) y `redis` (broker Celery).
> 2. Se ejecuta `airflow-init` (migra la base de datos interna, crea el usuario administrador `airflow` y termina con `Exited (0)`).
> 3. Se inician en segundo plano `airflow-apiserver`, `airflow-scheduler`, `airflow-dag-processor`, `airflow-worker` y `airflow-triggerer`, devolviendo inmediatamente el control de tu terminal.

3. Verificar que todos los contenedores estén `healthy`:

```
docker compose -f airflow/docker-compose.yaml ps
```

Deberás ver los 5 servicios principales de Airflow con estado `Up ... (healthy)` y `airflow-init` como `Exited (0)`.

4. Acceder al Panel Web de Apache Airflow:

- **URL:** `http://localhost:8080`
- **Usuario:** `airflow`

- **Contraseña:** `airflow`

Encontrarás únicamente los **2 DAGs del proyecto** listos y activos:

1. **`fraud_simulation_pipeline`:** Se ejecuta cada 5 minutos (`*/5 * * * *`). Simula un lote de 10 a 50 nuevas transacciones en `raw.transactions` y las evalúa con los modelos guardando los resultados en `raw.predictions`.
2. **`fraud_model_retraining_pipeline`:** Se ejecuta semanalmente (`@weekly`) para reentrenar y actualizar los artefactos `.joblib` con nuevos datos.

5. Comandos útiles de administración de Docker:

```
# Ver las últimas 50 líneas de logs de un servicio sin quedar atrapado:
docker compose -f airflow/docker-compose.yaml logs --tail 50 airflow-scheduler
docker compose -f airflow/docker-compose.yaml logs --tail 50 airflow-worker

# Detener el clúster conservando los datos:
docker compose -f airflow/docker-compose.yaml down

# Reiniciar el clúster desde cero limpiando volúmenes internos de Airflow:
docker compose -f airflow/docker-compose.yaml down -v
```

Paso 5: Ejecutar el Centro de Operaciones Web Interactivo (Streamlit)

Con tu entorno virtual activado en la raíz del proyecto, inicia la aplicación web interactiva `app.py`:

```
streamlit run app.py
```

- **URL local:** `http://localhost:8501`
- **Funcionalidades disponibles:**
  - Carga en 1 clic de escenarios predefinidos (*Transacción Legítima*, *Fraude Crítico Internacional R1*, *Anomalía Isolation Forest*, *Madrugada Sin Tarjeta R3* o *Personalizado*).
  - Evaluación en tiempo real de probabilidad de fraude, score de anomalía, índice de riesgo compuesto y reglas de negocio `R1`–`R5`.
  - Simulación visual del **SMS bancario enviado al cliente** y del **Webhook JSON** despachado al SOC/SIEM.
  - Monitoreo en vivo de las transacciones y predicciones registradas en PostgreSQL por Airflow.

Paso 6: Ejecución por Línea de Comandos (CLI) y Scoring por Lotes

A. Evaluar transacciones individuales por consola (`scripts/simulate_single_transaction.py`):

```
# Escenario de fraude crítico
python scripts/simulate_single_transaction.py --preset fraud
```

```
# Escenario de transacción legítima
python scripts/simulate_single_transaction.py --preset normal

# Escenario de anomalía de comportamiento
python scripts/simulate_single_transaction.py --preset anomaly

# Parámetros personalizados y salida en formato JSON
python scripts/simulate_single_transaction.py --amount 3500 --high-risk-country 1
--card-present 0 --json
```

B. Ejecutar inferencia por lotes sobre transacciones pendientes en PostgreSQL (`src/score_transactions.py`):

```
python src/score_transactions.py
```

🔧 4. Solución de Problemas Frecuentes en Linux / WSL2

1. **Los contenedores de Airflow en Linux no logran conectarse al PostgreSQL del host (`Connection refused` a `host.docker.internal:5432`):**
   - En Linux nativo, PostgreSQL suele escuchar únicamente en `localhost` (`127.0.0.1`). Para permitir que los contenedores de Docker se conecten a través de `host.docker.internal` (interfaz `docker0`, ej. `172.17.0.1`):
     1. En `/etc/postgresql/<version>/main/postgresql.conf`, asegúrate de tener:

        ```
        listen_addresses = '*'
        ```

     2. En `/etc/postgresql/<version>/main/pg_hba.conf`, agrega la subred de Docker:

        ```
        host    all             all             172.16.0.0/12
        scram-sha-256
        ```

     3. Reinicia PostgreSQL: `sudo systemctl restart postgresql`.
2. **Lentitud al escanear archivos en WSL2 (`/mnt/c/...`):**
   - Si ejecutas Ubuntu/WSL2 sobre una carpeta ubicada en el disco de Windows (`/mnt/c/Users/...`), el sistema de archivos 9P es más lento que el sistema de archivos nativo de Linux (`ext4`). La configuración actual ya optimiza `MIN_FILE_PROCESS_INTERVAL=60` y desactiva los DAGs de ejemplo para que funcione fluido incluso desde `/mnt/c/`.